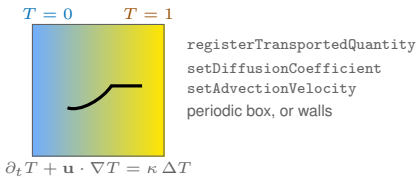
IBFE WITH ADVECTION–DIFFUSION

A passive scalar carried and spread by the flow round a moving body (IBFE/12): the equation, the two solvers IBAMR offers and what each needs, initial and boundary conditions that agree with each other, the jump the course's linear profile hid in a periodic box, and what the numbers say



Laura A. Miller

Department of Mathematics, University of Arizona

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. write down what a passive scalar is, what the advection–diffusion equation says, and when diffusion matters against the grid's own smearing (the Péclet number and $\sqrt{\kappa t}$);
2. add an advected and diffused quantity to any IBFE (or IB) program: the two headers, the solver object, the transported variable with its diffusion coefficient, initial condition, boundary conditions and advection velocity, and the one line that couples it to the fluid solver;
3. choose between IBAMR's semi-implicit and predictor–corrector solvers for the scalar, write their input blocks, and know the one extra line the predictor–corrector needs with an Adams–Bashforth fluid solver;
4. give the scalar initial and boundary conditions that agree with each other and with the box: periodic profiles in a periodic box, Robin conditions on walls, and the face numbering;
5. explain what the course's linear profile did in a periodic box, measure it, and fix it;
6. run IBFE/12, read its numbers, and look at T with the moving plate in VisIt.

> One more equation, solved on the same grid by the same time integrator, with the velocity the IB method just computed. The structure never notices.

WHERE THIS CHAPTER COMES FROM

- "8-IBFE with Advection and Diffusion" (July 2017; 14 slides): the half-tethered plate of chapter 17 moving through "a temperature (or concentration) gradient" that "begins as a linear function of x with periodic boundary conditions"; four VisIt snapshots; the initial and boundary condition blocks; a review of Robin conditions and IBAMR's face numbering; the `main.C` and `input2d` additions, block by block.
- "9-IBFE with Advection and Diffusion" (2020; 18 slides; `IBFE_Lecture_9`): the same deck pointing at the course example `9-IBFE_TetherForceHalfPlate2D_advdiff`, plus a last slide on rescaling the tether stiffness and the elastic modulus. The MATH 769 (2018) version is the same material.
- The 2016 example zip `Example_IBFE_TetherForceHalfPlate2D_advdiff_2016-11`: the course program and input the slides quote.
- Example IBFE/12 (this series): the course program revised and run with IBAMR 0.16.0 and 0.19.0 (semi-implicit and predictor–corrector; the course's initial profile; walls; a larger κ), with the README's measured tables.
- IBAMR 0.19.0: `AdvDiffSemiImplicitHierarchyIntegrator`, `AdvDiffPredictorCorrectorHierarchyIntegrator` and their input keys; `examples/IB/explicit/ex7` (a point-method body with a dye) and `examples/adv_diff/` (the scalar solvers on their own).
- New here: the equation and its scales; the solvers compared on the same run; the course input's solver line that was never read and the first-step error that follows; the jump the linear profile makes where the periodic box wraps, measured, and the periodic profile that replaces it; the figures from the shipped runs; the rescaling slide put in context.

THE EQUATION, AND ITS SCALES

A *passive* scalar T (a temperature, a concentration, a dye, oxygen) is carried by the velocity $\mathbf{u}$ the IB method computes and spreads by diffusion:

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \kappa \, \Delta T.$$

- Passive: T does not act back on the fluid or the body. One-way coupling: the fluid solver advances $\mathbf{u}$, then the scalar solver advances T with that $\mathbf{u}$. (A scalar that changes the density or drives a force is a different, two-way problem.)
- Advection ($\mathbf{u} \cdot \nabla T$) moves the field with the fluid; diffusion ($\kappa \Delta T$) smooths it. Their ratio over a length L at speed U is the Péclet number $\mathrm{Pe} = UL/\kappa$.
- The diffusion length in a time t is $\sqrt{\kappa t}$: how far a sharp edge spreads.

In IBFE/12: $U = 0.5$ (the tether's speed), $L = 1$ (the plate), $\kappa = 10^{-6}$:

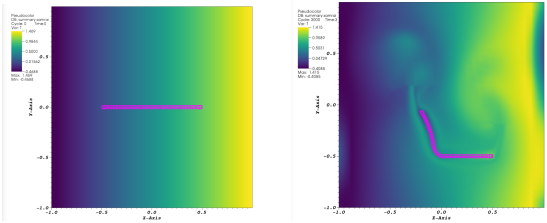
$\mathrm{Pe} = UL/\kappa$	5×10^5: advection dominates
$\sqrt{\kappa t}$ at $t = 4$	0.002: less than one fine cell (0.0156)
with $\kappa = 10^{-3}$	0.063: four fine cells

- With $\kappa = 10^{-6}$ the physical diffusion is invisible on this grid; what smooths T is the advection scheme itself ("numerical diffusion"), and the choice of scheme is what you see (slide 21).
- Water: heat $\kappa \approx 1.4 \times 10^{-3}$ cm^2/s, salt $\approx 1.5 \times 10^{-5}$, oxygen $\approx 2 \times 10^{-5}$: a dye in a 1 cm box at 0.5 cm/s is at $\mathrm{Pe} \sim 10^4$, and the same remark applies.

THE EXAMPLE: THE HALF-TETHERED PLATE STIRS A FIELD

"This example includes a flexible plate that moves up then down that is tethered to target points on the one half of its length. A temperature (or concentration) gradient is considered that begins as a linear function of x with periodic boundary conditions" (2020, slide 2).

box	2×2, periodic in x and y; two levels, $32 \to 128$ cells, $\mathrm{DX} = 1/64$
fluid	$\rho = 1$, $\mu = 0.01$
plate	chapter 17's `IBFE_Mesh2D_128.mat`: 1×0.0312, 101 nodes, 130 triangles; neo-Hookean, $\mu_s = 10$
tether	the right half ($s_0 > 0$), $\kappa_s = 10^6$; the target moves up at 0.5 until $t = 1$, down until $t = 3$, then releases (IBFE/01, `MOTION = "HALF"`)
T	$\kappa = 10^{-6}$; starts at 0 on the plate's left end and 1 on its right end
time	`DT = 1e-3`, 4000 steps to $t = 4$, 41 frames; about 90 s on four cores



Two of the 2020 deck's four snapshots (slide 3): T at $t = 0$ and $t = 3$ with the course's linear profile. Read the colour bar: T runs from -0.47 to 1.47, not from 0 to 1 (slide 17).

Snapshots from the 2020 deck (Laura Miller, VisIt).

THE PROGRAM (1): HEADERS AND THE SOLVER FOR T

```
// Headers for application-specific algorithm/data structure objects
#include <ibamr/AdvDiffPredictorCorrectorHierarchyIntegrator.h>
#include <ibamr/AdvDiffSemiImplicitHierarchyIntegrator.h>
...
// after the fluid solver, IBFEMethod and the IB time integrator exist (chapter 23, block 7):
Pointer<AdvDiffHierarchyIntegrator> adv_diff_integrator;
const string adv_diff_solver_type = input_db->getStringWithDefault("ADV_DIFF_SOLVER_TYPE", "SEMI_IMPLICIT");
if (adv_diff_solver_type == "PREDICTOR_CORRECTOR")
{
    Pointer<AdvectorExplicitPredictorPatchOps> predictor = new AdvectorExplicitPredictorPatchOps(
        "AdvectorExplicitPredictorPatchOps", app_initializer->getComponentDatabase("AdvectorExplicitPredictorPatchOps"));
    adv_diff_integrator = new AdvDiffPredictorCorrectorHierarchyIntegrator(
        "AdvDiffPredictorCorrectorHierarchyIntegrator",
        app_initializer->getComponentDatabase("AdvDiffPredictorCorrectorHierarchyIntegrator"), predictor);
}
else if (adv_diff_solver_type == "SEMI_IMPLICIT")
{
    adv_diff_integrator = new AdvDiffSemiImplicitHierarchyIntegrator(
        "AdvDiffSemiImplicitHierarchyIntegrator", app_initializer->getComponentDatabase("AdvDiffSemiImplicitHierarchyIntegrator"));
}
else
{
    TBOX_ERROR("Unsupported ADV_DIFF_SOLVER_TYPE: " << adv_diff_solver_type << "\n" << "Valid options are: PREDICTOR_CORRECTOR,
  SEMI_IMPLICIT");
}
pout << "Advection-diffusion solver: " << adv_diff_solver_type << "\n";
navier_stokes_integrator->registerAdvDiffHierarchyIntegrator(adv_diff_integrator);
```

THE PROGRAM (1A): WHAT THE SOLVER BLOCK DOES

- “After you create the major algorithm and data objects that comprise the application (e.g. setting the solver_type, IBFE method, etc.) add the following for the adv diff solver” (2020, slide 13). The last line is the coupling: the fluid integrator now owns a scalar integrator and advances it inside every one of its own steps, with the velocity it has just computed.
- `AdvDiffHierarchyIntegrator` is the base class of both solvers, as `INSHierarchyIntegrator` is of the two fluid solvers (chapter 23).

THE PROGRAM (1B): THE SOLVER LINE THE COURSE INPUT NEVER REACHED

The 2017 and 2020 slides read the choice from the `Main` block:

```
Pointer<Database> main_db = app_initializer->getComponentDatabase("Main");
const string adv_diff_solver_type = main_db->getStringWithDefault("adv_diff_solver_type", "PREDICTOR_CORRECTOR");
```

while the input defined `ADV_DIFF_SOLVER_TYPE = "SEMI_IMPLICIT"` at the top (slide 14) and never put `adv_diff_solver_type = ADV_DIFF_SOLVER_TYPE` into `Main`: the slides did not show that line.

- So a program and input copied from the slides use the *default*, `PREDICTOR_CORRECTOR`, whatever the top of the input says, and stop at the first step (slide 13). Tested with the course program and IBAMR 0.19.0.
- IBFE/12 reads `ADV_DIFF_SOLVER_TYPE` from the top level directly, defaults to `SEMI_IMPLICIT`, and prints its choice at the start of the run (“`Advection-diffusion solver:  SEMI_IMPLICIT`”), so the log says which solver ran.
- Chapter 23’s rule again: a key is read only where the program looks for it, and the log’s `// input used` marks tell you.

THE PROGRAM (2): THE TRANSPORTED QUANTITY

```
const bool periodic_domain = grid_geometry->getPeriodicShift().min() > 0;
// Setup the advected and diffused quantity.
Pointer<CellVariable<NDIM, double> > T_var = new CellVariable<NDIM, double>("T");
adv_diff_integrator->registerTransportedQuantity(T_var);
adv_diff_integrator->setDiffusionCoefficient(T_var, input_db->getDouble("KAPPA"));
Pointer<CartGridFunction> T_init = new muParserCartGridFunction(
    "T_init", app_initializer->getComponentDatabase("TemperatureInitialConditions"), grid_geometry);
adv_diff_integrator->setInitialConditions(T_var, T_init);
std::unique_ptr<RobinBcCoefStrategy<NDIM> > T_bc_coef;
if (!periodic_domain)
{
    T_bc_coef = std::make_unique<muParserRobinBcCoefs>(
        "T_bc_coef", app_initializer->getComponentDatabase("TemperatureBcCoefs"), grid_geometry);
    adv_diff_integrator->setPhysicalBcCoef(T_var, T_bc_coef.get());
}
adv_diff_integrator->setAdvectionVelocity(T_var, navier_stokes_integrator->getAdvectionVelocityVariable());
```

- “Before you Configure the IBFE solver in main.C, add these lines (note they go after the pointers for load balancers and gridding algorithms)” (2020, slide 12): block 8 of chapter 23’s map, before block 9.
- Five calls, in this order: the variable (a cell-centred field named `T`, which is the name VisIt will show), its diffusion coefficient, its initial condition (a muParser block, like the velocity’s), its boundary conditions (only if the box is not periodic; a periodic box overrides them), and the velocity that carries it, which is the fluid solver’s own advection velocity.
- The 2020 code held the boundary object in a raw pointer set to `NULL` and never deleted it; IBFE/12 uses a `unique_ptr`. A second scalar is five more lines with another name.

THE INPUT (1): THE KEYS AND THE TWO SOLVER BLOCKS

```
KAPPA = 1.0e-6                    // temperature diffusion
  coefficient
...
ADV_DIFF_SOLVER_TYPE         = "SEMI_IMPLICIT"    // or
  PREDICTOR_CORRECTOR
ADV_DIFF_NUM_CYCLES          = 2                  // fixed-point
  cycles
ADV_DIFF_CONVECTIVE_TS_TYPE = "MIDPOINT_RULE"    // time stepping
ADV_DIFF_CONVECTIVE_OP_TYPE = "PPM"              // spatial scheme
ADV_DIFF_CONVECTIVE_FORM     = "ADVECTIVE"        // or
  CONSERVATIVE
DT_MAX                       = 0.1                // its own step
  limit
...
AdvDiffSemiImplicitHierarchyIntegrator {
   start_time                    = START_TIME
   end_time                      = END_TIME
   grow_dt                       = GROW_DT
   num_cycles                    = ADV_DIFF_NUM_CYCLES
   convective_time_stepping_type = ADV_DIFF_CONVECTIVE_TS_TYPE
   convective_op_type            = ADV_DIFF_CONVECTIVE_OP_TYPE
   convective_difference_form    = ADV_DIFF_CONVECTIVE_FORM
   cfl                           = CFL_MAX
   dt_max                        = DT_MAX
   tag_buffer                    = TAG_BUFFER
   enable_logging                = ENABLE_LOGGING
}
```

```
AdvDiffPredictorCorrectorHierarchyIntegrator {
   start_time                 = START_TIME
   end_time                   = END_TIME
   grow_dt                    = GROW_DT
   convective_difference_form = ADV_DIFF_CONVECTIVE_FORM
   cfl                        = CFL_MAX
   dt_max                     = DT_MAX
   tag_buffer                 = TAG_BUFFER
   enable_logging             = ENABLE_LOGGING
   AdvDiffPredictorCorrectorHyperbolicPatchOps {
      compute_init_velocity  = TRUE
      compute_half_velocity  = TRUE
      compute_final_velocity = FALSE
      extrap_type = "LINEAR"
   }
   HyperbolicLevelIntegrator {
      cfl                       = CFL_MAX
      cfl_init                  = CFL_MAX
      lag_dt_computation        = TRUE
      use_ghosts_to_compute_dt = FALSE
   }
}
AdvectorExplicitPredictorPatchOps {
   limiter_type = "MC_LIMITED"
}
```

“Add these to the bottom of the list of solver parameters . . . Add a diffusion coefficient to the physical parameters . . . Add settings for advection diffusion solver” (2020, slides 14–17). Both blocks are in the input, so the switch works; only the chosen one is read.

THE INPUT (2): WHAT THE KEYS MEAN, AND THEIR ALIASES

key	meaning (and the spellings `getFromInput` accepts)
`num_cycles`	fixed-point iterations per step of the semi-implicit solver; 2 here, the fluid solver's own number
`convective_time_stepping_type`	how the advection term is stepped: `MIDPOINT_RULE`, `ADAMS_BASHFORTH`, `FORWARD_EULER`, `TRAPEZOIDAL_RULE`; also `convective_timestepping_type`, `default_convective_time_stepping_type`
`init_convective_time_stepping_type`	the same for the first step, when there is no previous step to extrapolate from (slide 13)
`convective_op_type`	the spatial scheme: `PPM` (piecewise parabolic), `CENTERED`, `CUI`, `WAVE_PROP`; also `convective_operator_type`
`convective_difference_form`	`ADVECTIVE` ($\mathbf{u} \cdot \nabla T$) or `CONSERVATIVE` ($\nabla \cdot (\mathbf{u} T)$); also `_type`
`diffusion_time_stepping_type`	how $\kappa \Delta T$ is stepped (implicit; the trapezoidal rule by default)
`cfl`, `cfl_max`, `CFL`, `CFL_max`	the scalar solver's own CFL limit on its step
`dt_max`	its own step limit, which the fluid's step then also bounds: `DT_MAX = 0.1` does nothing here against `DT = 1e-3`
`limiter_type` (predictor)	the slope limiter of the Godunov predictor: `MC_LIMITED`, `MINMOD_LIMITED`, `SUPERBEE_LIMITED`, `NONE`

- The scalar solver's step is the fluid's step: `registerAdvDiffHierarchyIntegrator` makes the fluid integrator call it once per fluid step (or per cycle). Its own `dt_max` and `cfl` only matter if they are *smaller* than the fluid's.
- `tag_buffer`, `start_time`, `end_time`, `grow_dt` repeat the fluid's values; they must not disagree.

THE TWO SOLVERS

	`SEMI_IMPLICIT`	`PREDICTOR_CORRECTOR`
advection	explicit, the fluid solver's own scheme (PPM with the midpoint rule here), with `num_cycles` fixed-point sweeps	an explicit Godunov predictor (the class `AdvectorExplicitPredictorPatchOps`): upwind with a slope limiter, the half-step velocity, corner transport
diffusion	implicit (trapezoidal rule)	implicit
overshoot	can overshoot a little where the field folds (1.008 and -0.003 on a field between 0 and 1)	none: the limiter keeps T within its starting range; the extremes wear down instead ($0.9998 \to 0.9965$ by $t = 4$)
first step	no requirement	one cycle only: the fluid solver's first step must be single-cycle too (slide 13)
when	the default; the same scheme as the fluid, one set of settings	a bounded scalar (a concentration that must stay in $[0, 1]$), or a sharp front

- Both solve $\partial_t T + \mathbf{u} \cdot \nabla T = \kappa \Delta T$ on the same adaptive grid with the velocity the fluid solver provides; the difference is the advection scheme, and at $\mathrm{Pe} = 5 \times 10^5$ that difference is what you see (slide 21).
- `examples/adv_diff/` in IBAMR runs the two solvers without any body: `ex0`–`ex8`, with analytic solutions to compare against, are where to go when a scalar result looks wrong.

THE PREDICTOR–CORRECTOR'S ONE EXTRA LINE

With `ADV_DIFF_SOLVER_TYPE = "PREDICTOR_CORRECTOR"` and nothing else changed, the run stops at the first step:

```
INSStaggeredHierarchyIntegrator::preprocessIntegrateHierarchy():
  attempting to perform 2 cycles of fixed point iteration.
  number of cycles required by coupled advection-diffusion solver = 1.
  current implementation requires either that both solvers use the same number of cycles,
  or that the Navier-Stokes solver use only a single cycle.
```

- The fluid solver steps its convective term with Adams–Bashforth, which has no previous step at $t = 0$ and so takes *two* cycles for the first step; the predictor–corrector scalar solver can take only one. The two must agree.
- The fix is one line in the `INSStaggeredHierarchyIntegrator` block, shipped commented out in IBFE/12's input:

```
INSStaggeredHierarchyIntegrator {
   ...
   // init_convective_time_stepping_type = "FORWARD_EULER"   // remove the // for PREDICTOR_CORRECTOR
}
```

- The first fluid step then uses forward Euler for its convective term, a single cycle, and the rest of the run is unchanged (Adams–Bashforth from the second step on).
- This is the error the course's slides would have produced on current IBAMR, through the unread key of slide 8: the course program defaulted to the predictor–corrector, and the input had no single-cycle first step.

INITIAL AND BOUNDARY CONDITIONS FOR T

```
TemperatureInitialConditions {
   // the course's profile, 0.5 + x: see the next slides
   // function = "0.5+X_0"
   // two of the course's other trials (they jump where the box
  wraps in y)
   // function = "0.5*(tanh(80.0*(-X_1-0.15))+1)"
   // function = "(0.5*(tanh(80.0*(-X_1-0.15))+1))*(0.5*(tanh
  (80.0*(X_0+0.115))+1))*(0.5*(tanh(80.0*(-X_0+0.115))+1))"
   L = L
   function = "0.5 + 0.5*sin(2*PI*X_0/L)"
}
TemperatureBcCoefs {   // read only on sides that are not
  periodic
   L = L
   acoef_function_0 = "1.0"   ...   acoef_function_3 = "1.0"
   bcoef_function_0 = "0.0"   ...   bcoef_function_3 = "0.0"
   gcoef_function_0 = "0.5 + 0.5*sin(2*PI*X_0/L)"   ...
  gcoef_function_3 = "0.5 + 0.5*sin(2*PI*X_0/L)"
}
```

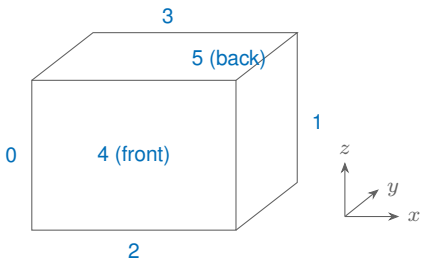
“The boundary conditions are set up the same way as the velocity boundary conditions (e.g. periodic or Robin boundary conditions). In this example, the boundary conditions are periodic (overrides Robin) and the initial concentration is a linear function of x” (2020, slide 4).

- The initial condition is any muParser expression of `X_0`, `X_1` (and `X_2`) and the block’s own constants; chapter 7’s rules for the velocity’s blocks apply, including that a block sees only its own constants (`L = L` copies it in).
- Periodic overrides Robin: on a periodic side the block is never read; the two sides are the same place.

INITIAL AND BOUNDARY CONDITIONS (2): ROBIN CONDITIONS

- The boundary condition is Robin on each face: $a\,T + b\,\partial T/\partial n = g$. “Robin boundary conditions are a weighted combination of Dirichlet boundary conditions and Neumann boundary conditions” (2020, slide 8). $a = 1$, $b = 0$: $T = g$ on the face (a wall held at a temperature); $a = 0$, $b = 1$: $\partial T/\partial n = g$ (an insulated wall for $g = 0$; a prescribed flux otherwise); both non-zero: a wall that exchanges heat with its surroundings at a rate set by b/a.
- The sign of the normal: “In one dimension, if $\Omega = [0, 1]$, the Robin boundary condition becomes $au(0) - bu'(0) = g(0)$, $au(1) + bu'(1) = g(1)$. Notice the change of sign . . . because the normal at $x = 0$ points in the negative direction” (slide 9). IBAMR’s $\partial/\partial n$ is the outward normal derivative on every face, so the input never carries that sign: $b = 1$, $g = 0$ is “no flux” on any face.
- The velocity’s conditions (chapter 7) are the same objects with the same keys, one block per component; a scalar needs one block.
- Initial and boundary values must agree where they meet: a wall held at $T = g$ next to an initial field with a different value there starts the run with a jump at the wall, the same trouble as slide 17 in another place.

THE FACES, NUMBERED



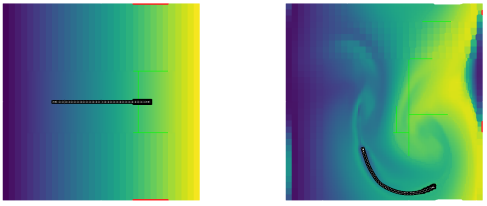
Redrawn from the 2020 deck's slide 10 (a box of unknown origin with the faces numbered).

"IBAMR boundary numbering. Look in the input2D/3D files. 2D: xleft = 0, xright = 1, ybottom = 2, ytop = 3. 3D: add zbottom = 4, ztop = 5" (2020, slide 10).

- The rule: face $2d$ is the low side of direction d and face $2d+1$ the high side; $d = 0, 1, 2$ for x, y, z. The same numbering serves `VelocityBcCoefs_d`, `TemperatureBcCoefs`, the sponge layer of chapter 23 and the output of the `SpongeLayerForceFunction` keys.
- The `_d` in `VelocityBcCoefs_d` is the velocity *component*; the `_0` to `_5` in `gcoef_function_0` is the *face*. A scalar has one block with six faces; a velocity has three blocks with six faces each.
- In the chapter 22 tank the x-velocity was set on all six faces (`gcoef_function_0` to `_5` of `VelocityBcCoefs_0`); here, with periodicity on, no face is read at all.

THE JUMP THE LINEAR PROFILE HID

- “The temperature is a linear function of x which has an initial range of -0.5 to 1.5” (2020, slide 5): $T = 0.5 + x$ on $-1 < x < 1$ gives 0 at the plate’s left end and 1 at its right end, which was the intent.
- But the box is periodic: $x = 1$ touches $x = -1$. The profile is 1.5 on one side of that seam and -0.5 on the other, a jump of 2 at the edge of the box, carried into the fluid as soon as anything moves there. The colour bar on the course’s snapshots (-0.47 to 1.47) is this jump.
- Measured (the course program, IBAMR 0.19.0): in the coarse cell next to the right side of the box at mid-height ($x = 0.97$, $y = 0.03$), T starts at 1.47 and falls to -0.16 by $t = 4$: fluid that leaves through the left side, where $T \approx -0.5$, comes back in through the right. The average of T over the box, which should stay 0.5, falls to 0.495.



The course’s $T = 0.5 + x$ at $t = 0$ and $t = 4$ (colour from -0.5 to 1.5): the seam at the left and right edges is where 1.5 meets -0.5.

In a periodic box the initial field must be periodic too: same value, and ideally the same slope, on the two sides of every periodic direction. The same holds for the velocity and for any forcing.

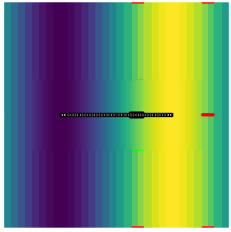
THE JUMP (2): TWO FIXES

Keep the box periodic, make the profile periodic (IBFE/12):

$$T = \tfrac{1}{2} + \tfrac{1}{2}\sin\left(\frac{2\pi x}{L}\right), \qquad L = 2,$$

which is 0, $\frac{1}{2}$ and 1 at $x = -\frac{1}{2}$, 0 and $\frac{1}{2}$, the three values the course formula had on the plate, and $\frac{1}{2}$ with the same slope at both edges of the box. T stays between 0 and 1 and its average stays $\frac{1}{2}$ (slide 19).

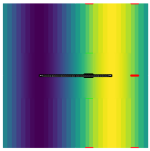
Or put walls round the box and hold T on them: `periodic_dimension = 0,0`, no-slip velocity on the four faces, and `TemperatureBcCoefs` with $a = 1$, $b = 0$, g = the initial profile, so that the initial and boundary values agree on the walls. Any profile then works, the linear one included.



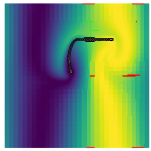
IBFE/12's periodic profile at $t = 0$ (colour from 0 to 1).

- With walls the plate moves differently (by up to 0.15 at $t = 4$: the walls are only one plate-length away), and T stays within -0.033 and 1.015 (slide 20).
- The course's two other trials, `tanh` profiles in y (kept as comments in the input), jump where the box wraps in y; the same rule, the other direction.

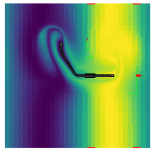
WHAT YOU SHOULD SEE (1): THE PLATE FOLDS THE FIELD



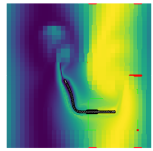
$t = 0$



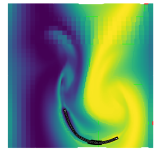
$t = 1$: the tethered half has risen 0.5



$t = 2$: on the way down



$t = 3$: released



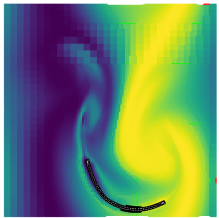
$t = 4$: coasting

- T from 0 (dark) to 1 (yellow); the plate in black, displaced by `dx`. The free half trails the tethered half (chapter 17), and the field is folded by the flow the plate drives: the stripes are the initial profile, stretched and wrapped.
- The plate's motion is IBFE/01's to 2×10^{-8}: the scalar changes nothing on the fluid or the body.
- Each place is counted once, on the finest grid that covers it (chapter 29's queries):

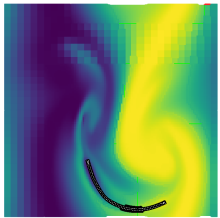
t	0	1	2	4
smallest T	0.0002	-0.0031	-0.0012	-0.0006
largest T	0.9998	1.0003	1.0000	1.0053
average T	0.50000	0.49995	0.50001	0.49992

IBFE/12, IBAMR 0.19.0 (`/root/work/0.19.0/ibfe12_final`); `wsl/final/visit_ch24.sh`. 0.16.0 gives the same numbers.

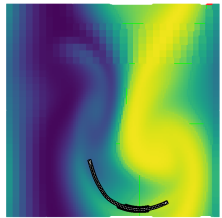
WHAT YOU SHOULD SEE (2): THE VARIANTS AT $t = 4$



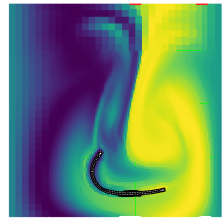
semi-implicit, $\kappa = 10^{-6}$



predictor–corrector, $\kappa = 10^{-6}$



semi-implicit, $\kappa = 10^{-3}$



walls instead of periodic sides

predictor–corrector	never above the starting maximum (0.9998) nor below -0.0003; average within 0.4999–0.5003; the largest T falls to 0.9965 by $t = 4$: no overshoot, slightly worn extremes
$\kappa = 10^{-3}$	diffusion over $\sqrt{\kappa t} = 0.06$, four fine cells, by $t = 4$: the stripes blur; T between 0.017 and 0.979; no overshoot; average within 10^{-4} of 0.5
walls	`periodic_dimension = 0,0`: the fluid sticks to the walls and T is held at its initial values there; the plate moves differently (up to 0.15 at $t = 4$); T between -0.033 and 1.015

WHAT YOU SHOULD SEE (3): THE SOLVERS SIDE BY SIDE

- At $\kappa = 10^{-6}$ the two solvers give the same large-scale picture (the stripes, the fold round the plate) and differ in the small things: the semi-implicit scheme overshoots by 0.5% at the folds (1.008 at worst over all frames) and undershoots by 0.3%; the predictor–corrector's limiter forbids both, and pays with extremes that decay by 0.35% in four time units. Neither is "right": the exact solution at this Péclet number has stripes thinner than a cell, which no scheme on this grid can hold.
- $\kappa = 10^{-3}$ is the honest case for this grid: the physical diffusion (4 cells) exceeds the numerical one, both solvers agree closely, and the result converges under refinement (chapter 16's test).
- For a quantity that must stay bounded (a concentration, a volume fraction, a probability), use the predictor–corrector, or clip in the post-processing and say so.
- The run costs the same either way: 91 s against IBFE/01's 70 s on four cores with 0.19.0; the scalar solve is cheap next to the fluid's.

THE 2020 DECK'S LAST SLIDE: RESCALING THE TETHER AND THE MODULUS

"Rescaling kappa_spring and the elastic modulus. $F = kx$.

$$k_{\text{new}} = \left(k_{\text{old}}\,(F_{\text{new}}/F_{\text{old}})\right)(ds_{\text{old}}/ds_{\text{new}}), \qquad F \sim \rho L U^2.$$

Elastic modulus: try leaving it the same. Scale mu and beta with e" (2020, slide 18), followed by the `E_STEM`, `E_LEAF`, `MU_*`, `BETA_*` lines of chapter 22 with $E = 10^7$.

- The slide answers a question from the room, not the chapter: "I changed the units (or the grid) and the plate went limp (or exploded): how do I move the stiffnesses?"
- The tether. The tether force per point is $\kappa_s\,\Delta x$, and the force it must balance scales with the fluid's $\rho L U^2$ (chapter 8b). Change the problem's scale and κ_s scales with $F_{\text{new}}/F_{\text{old}}$; change the mesh spacing and the force per point must change with the number of points per length, $ds_{\text{old}}/ds_{\text{new}}$. Both factors are on the slide.
- The modulus. E (and μ_s, β with it) is a stress, force per area: it does not change with the mesh at all, and with the scale it changes like ρU^2. "Try leaving it the same" is right when the units are unchanged and only the grid is; otherwise scale it with the dynamic pressure, as chapter 19 (slide 17) did for μ_s.
- A cleaner route is chapter 8b's: write the input in terms of the dimensionless groups (the Reynolds number, $\kappa_s/(\rho U^2 L)$, $E/(\rho U^2)$, $\kappa/(UL)$ for the scalar) and let the file compute the constants, so that changing U, L or DX changes nothing you have to think about. IBFE/10's `KAPPA_S = 2.5e1*DX/(DT^2)` is the grid part of that rule written out.
- For the scalar itself there is nothing to rescale: κ is a physical constant, and its dimensionless form is $1/\mathrm{Pe}$.

RELATIVES: THE POINT METHOD, AND THE COURSE'S 2014 ARCHIVE

- Upstream `IB/explicit/ex7` does the same five calls for a point-method body: a dye `Q` with diffusion coefficient `D`, transported by the flow round an `IBMethod` structure. The scalar machinery does not care which IB method moves the body; chapter 15 ran it.
- The course's 2014 point-method examples (the `Advection-Diffusion-Examples-3803` folder of the archive): a peristaltic tube carrying a concentration, and a pulsing body that mixes one. Both are 2014 code for the 3803 build, not checked or ported in this series; their advection–diffusion lines are the same as here, with `IBMethod` in place of `IBFEMethod`. The tube is IB/22 in the example list, a candidate for a later port; the pulsing body is a research model and stays out.
- `examples/adv_diff/` in IBAMR: the scalar solvers alone, in a prescribed velocity field, with exact solutions: the place to check a scheme, a limiter or a boundary condition before it meets a body.
- Two-way coupling (a scalar that drives the flow: temperature through buoyancy, a concentration through a body force) is the same five calls plus a forcing function that reads `T`; IBAMR's `examples/navier_stokes` and the multiphase examples do it. Not covered here.

MEASURING THE SCALAR, AND LOOKING AT IT

In VisIt (the README's recipe):

1. Open `viz_IB2d/dumps.visit`; **Add** → **Pseudocolor** → **T**; **Draw**. `T` is there by its program name, next to `U`, `P`, `Omega`, `Div U` and `F`.
2. Open `viz_IB2d/output.ex2`; **Add** → **Mesh**; the expression `dX = {dX_0, dX_1}`; **Operators** → **Transforms** → **Displace** with `dX`.
3. Fix the colour range to 0–1 (**Pseudocolor** → **Min**/**Max**), or the overshoots and the jump of slide 17 rescale every frame.

The tables of slides 19–20 are VisIt queries (chapter 29): **Query** → **MinMax** on `T` for the extremes, **Weighted Variable Sum** divided by the box's area for the average, and a **Pick** at $(0.97, 0.03)$ for the course run's cell by the seam.

- `output_data` in IBFE/12 is chapter 23's state dump (the hierarchy's `U` and `P` to HDF5, the mesh and systems to libMesh files). To dump `T` with them, add its variable index to the `ComponentSelector`:

```
hier_data.setFlag(var_db->mapVariableAndContextToIndex(
    T_var, adv_diff_integrator->getCurrentContext()));
```

- A measurement in the program (the amount of T inside a region, the flux through a line) is chapter 11's loop over patches with `T`'s patch data; chapter 33 does the same from the files in Python.
- A scalar on the *body* (a temperature of the plate, a surface concentration) is a different object: a libMesh system on the mesh, chapter 19's `system_data`; IBAMR does not advect those.

WHAT CAN GO WRONG

symptom	cause and fix
“attempting to perform 2 cycles of fixed point iteration . . . coupled advection-diffusion solver = 1” at step 1	the predictor–corrector with an Adams–Bashforth fluid solver: `init_convective_time_stepping_type = "FORWARD_EULER"` (slide 13)
the log says the other solver than the input	the solver line is read from a block the input does not set (`Main`, slide 8); IBFE/12 prints its choice
T far outside its starting range near the box edge	a profile that is not periodic in a periodic box (slide 17); or walls with `gcoef` that disagree with the initial condition
`TemperatureBcCoefs` changed, no effect	the box is periodic in that direction; the block is read only on non-periodic faces
small over- and undershoots at folds	the semi-implicit scheme at high Péclet number; the predictor–corrector, or a larger κ, or a finer grid
T smears though κ is tiny	numerical diffusion: $\sqrt{\kappa t}$ is below a cell; refine, or accept and say so
the scalar’s `dt_max` or `cfl` seems ignored	the fluid’s step governs; the scalar’s limits only bite when smaller
a second scalar overwrites the first	the same variable name twice; each `CellVariable` needs its own name
“Unsupported ADV_DIFF_SOLVER_TYPE”	a spelling other than `SEMI_IMPLICIT` or `PREDICTOR_CORRECTOR`
the body’s own temperature	not this machinery: a libMesh system on the mesh (slide 24)

WHAT THE OLDER DECKS HAD THAT IS NOW PART X

2017–2020	now
`main_db->getStringWithDefault( "adv_diff_solver_type", "PREDICTOR_CORRECTOR")` with no such line in `Main`	`ADV_DIFF_SOLVER_TYPE` read at the top level, default `SEMI_IMPLICIT`, printed at start
`RobinBcCoefStrategy<NDIM>* T_bc_coef = NULL;` never deleted	a `std::unique_ptr`; `nullptr`
`T = 0.5 + x` in a periodic box	`0.5 + 0.5*sin(2*PI*X_0/L)`, and walls with matching `gcoef` as the alternative
the predictor–corrector without a single-cycle first step	`init_convective_time_stepping_type = "FORWARD_EULER"` ready in the input
`IBAMR_config.h`, `libMeshEnums::Order`, `MathUtilities<double>::equalEps`	chapter 17's list
"Temperature" for the block names with "(or concentration)" in the text	kept: the block names are the course's; the scalar is whatever you say it is
the rescaling slide with `E_STEM = 1.0e7` and no context	slide 22

EXERCISES

1. Run IBFE/12 with the course's `function = "0.5+X_0"` and watch T next to $x = \pm 1$ in VisIt with the colour range fixed to 0–1; then with the range free. Pick the cell at $(0.97, 0.03)$ at $t = 0$ and $t = 4$.
2. Switch to the predictor–corrector (with the extra line), and compare the largest T over the run with the semi-implicit solver's.
3. Put walls round the box and hold $T = 0$ on the left wall and $T = 1$ on the right ($a = 1$, $b = 0$, `gcoef_function_0 = "0.0"`, `_1 = "1.0"`, and $\partial T/\partial n = 0$ on the top and bottom): a plate stirring a box heated from one side.
4. $\kappa = 10^{-2}$: how long until the stripes are gone? Compare with $\sqrt{\kappa t} = L/2$.
5. Add a second scalar `C` with its own κ and a blob initial condition $\exp(-((x - 0.3)^2 + y^2)/0.01)$, and plot both.
6. Dump `T` in `output_data` (slide 24) and compute the average of T from the HDF5 file in Python (chapter 33); check it against the table of slide 19.
7. Add the five calls to IBFE/08 (chapter 21) and put a dye between the plate and the disc.

RULES OF THUMB

- Five calls per scalar, one line to couple it; the body never knows.
- Periodic box, periodic profile; walls, a boundary value that equals the initial value on the wall.
- $\sqrt{\kappa t}$ against the cell size tells you whether the diffusion you see is physical or the scheme's.
- The semi-implicit solver by default; the predictor–corrector, with the single-cycle first step, when the scalar must stay bounded.
- The scalar's step is the fluid's step; its own limits only bite when smaller.
- Fix the colour range before you judge a scalar picture.
- Stiffnesses scale with $\rho L U^2$ and with the points per length; moduli with ρU^2 only.

SUMMARY

1. A passive scalar obeys $\partial_t T + \mathbf{u} \cdot \nabla T = \kappa \Delta T$ with the IB method's velocity and does not act back; at $\mathrm{Pe} = 5 \times 10^5$ the grid's smearing, not κ, sets what you see.
2. Adding one to an IBFE program: two headers, a solver object registered with the fluid integrator, and for each scalar a cell variable, its κ, initial condition, boundary conditions (non-periodic faces only) and advection velocity.
3. The semi-implicit solver shares the fluid's scheme and overshoots slightly at folds; the predictor–corrector is bounded, wears its extremes, and needs the fluid's first step to be single-cycle.
4. The course's $T = 0.5 + x$ jumped by 2 where the periodic box wraps, and its solver line was never read; IBFE/12 uses a periodic profile, reads and prints the solver, and ships the single-cycle line.
5. The run takes 90 s; T stays between 0 and 1 with average 0.5; the variants (predictor–corrector, $\kappa = 10^{-3}$, walls) behave as the scales predict.

Chapter 25 tours IBAMR's own IBFE examples, `ex0` to `ex11`, with the map of chapter 23 in hand.

REFERENCES AND CREDITS

- Example IBFE/12 of this series (`main.cpp`, `input2d`, `IBFE_Mesh2D_128.mat`, README with the measured tables); the runs `ibfe12_final`, `ibfe12_predcorr_fe`, `ibfe12_kappa1e-3`, `ibfe12_walls` and the course program's `orig_ad_run` with IBAMR 0.19.0, and `ibfe12_default` with 0.16.0; `wsl/final/visit_ch24.sh`. Figures: VisIt 3.5.
- The course material: *8-IBFE with Advection and Diffusion* (July 2017) and *9-IBFE with Advection and Diffusion* (2020; its snapshots and box diagram), the MATH 769 version (2018), and the example zip `Example_IBFE_TetherForceHalfPlate2D_advdiff_2016-11`.
- IBAMR 0.19.0: `src/adv_diff/AdvDiffHierarchyIntegrator.cpp`, `AdvDiffSemiImplicitHierarchyIntegrator.cpp`, `AdvDiffPredictorCorrectorHierarchyIntegrator.cpp`, `src/advect/AdvectorExplicitPredictorPatchOps.cpp` (the input keys); `examples/IB/explicit/ex7`; `examples/adv_diff/ex0`–`ex8`. https://github.com/IBAMR/IBAMR
- Robin boundary conditions: the 2020 deck's slides 8–9 adapted the standard definition (as on Wikipedia's "Robin boundary condition"); the one-dimensional sign remark is theirs.